

Faza 1: Cuantizare Statică FP16

Evaluare ViT-Tiny pretrained

pe ImageNet-1k validation

Post-Training Quantization — FP32 vs. FP16

Student:

Tudose Alexandru

Grupa 10LF333

An III, Informatică Aplicată

Coordonator:

Conf. dr. ing. Honorius Gâlmeanu

Informatică Aplicată

Cuprins

1	Introducere	2
2	Metodologie	2
2.1	Model și date	2
2.2	Preprocessing	2
2.3	Protocol de evaluare	3
2.4	Metrici	3
3	Rezultate	3
3.1	Comparație globală	3
3.2	Acuratețe	4
3.3	Memorie	5
3.4	Latență	6
3.5	Tradeoff acuratețe–latență	7
4	Discuție	7
4.1	Validarea ipotezei	7
4.2	Comparație cu studiul preliminar	7
4.3	Implicații pentru fazele ulterioare	8
4.4	Limitări	8
5	Concluzii	8

1 Introducere

Acest raport documentează **Faza 1** a lucrării de licență, în care evaluăm impactul cuantizării statice FP16 asupra unui Vision Transformer pretrained. Spre deosebire de studiul preliminar (Faza 0), care a antrenat modelul pe CIFAR-10, această fază evaluează cuantizarea *post-training* — fără nicio re-antrenare — pe setul de validare ImageNet-1k.

Obiective:

1. Stabilirea unui **baseline FP32** pe ImageNet-1k validation (50 000 imagini, 1 000 clase).
2. Evaluarea conversiei `model.half()` (FP16 static) din punctul de vedere al acurateții, latenței și memoriei.
3. Cuantificarea tradeoff-ului precizie–eficiență pentru FP16 pe un model pretrained real.

Context: Cuantizarea FP16 este cea mai simplă formă de reducere a preciziei numerice. Conversia `model.half()` transformă toate ponderile din `float32` (4 bytes/param) în `float16` (2 bytes/param), reducând teoretic memoria la **exact jumătate**. Pe hardware modern (GPU-uri NVIDIA cu Tensor Cores, Apple Silicon cu suport nativ float16), operațiile FP16 pot fi și mai rapide decât FP32.

2 Metodologie

2.1 Model și date

Tabela 1: Configurația experimentală

Parametru	Valoare
Model	<code>vit_tiny_patch16_224</code> (timm)
Parametri	5.7 milioane
Pretrained	ImageNet-1k (<code>pretrained=True</code>)
Dataset evaluare	ImageNet-1k validation (50 000 imagini, 1 000 clase)
Batch size	64
Device	Apple M4, backend MPS
PyTorch	2.9.1

2.2 Preprocessing

Transformarea de intrare este cea specificată de modelul timm:

- Resize + center crop la 224×224 (crop ratio = 0.9)
- Interpolare bicubică
- Normalizare: $\mu = (0.5, 0.5, 0.5)$, $\sigma = (0.5, 0.5, 0.5)$

Notă: `vit_tiny_patch16_224` din timm nu folosește normalizarea standard ImageNet ($\mu = (0.485, 0.456, 0.406)$), ci $(0.5, 0.5, 0.5)$ — acesta este un aspect important, deoarece utilizarea normalizării greșite ar degrada semnificativ acuratețea.

2.3 Protocol de evaluare

1. **FP32 baseline:** Modelul pretrained este încărcat și evaluat direct în precizie `float32`.
2. **FP16:** Un al doilea model pretrained identic este convertit cu `model.half()`, care transformă *toate* ponderile și buffer-urile la `float16`.
3. Ambele modele sunt evaluate pe aceleași 50 000 de imagini, cu aceleași transformări de intrare.
4. **Warmup:** Primele 3 batch-uri sunt excluse din măsurătorile de latență, pentru a elimina overhead-ul de inițializare (compilare kernele MPS, cache-uri).
5. **Sincronizare:** După fiecare batch se apelează `torch.mps.synchronize()` pentru a asigura măsurători de latență corecte (operațiile MPS sunt asincrone).

2.4 Metrici

- **Top-1 Accuracy (%)**: Procentul de imagini clasificate corect.
- **Average Loss**: Cross-entropy loss mediu pe toate batch-urile.
- **Latency (ms/batch)**: Timp mediu de inferență per batch de 64 imagini, exclusiv warmup.
- **Model Memory (MB)**: Memoria ocupată de parametrii modelului, calculată ca $\sum p.\text{numel()} \times p.\text{element_size()}$.

3 Rezultate

3.1 Comparație globală

Tabela 2: Rezultate FP32 vs. FP16 pe ImageNet-1k validation

Format	Accuracy (%)	Loss	Latency (ms)	Memory (MB)
FP32 (baseline)	75.456	0.9420	164.96	21.81
FP16 (<code>model.half()</code>)	75.452	0.9420	135.14	10.91
Diferență	−0.004 pp	< 0.001	1.22 × speedup	2.00 × reducere

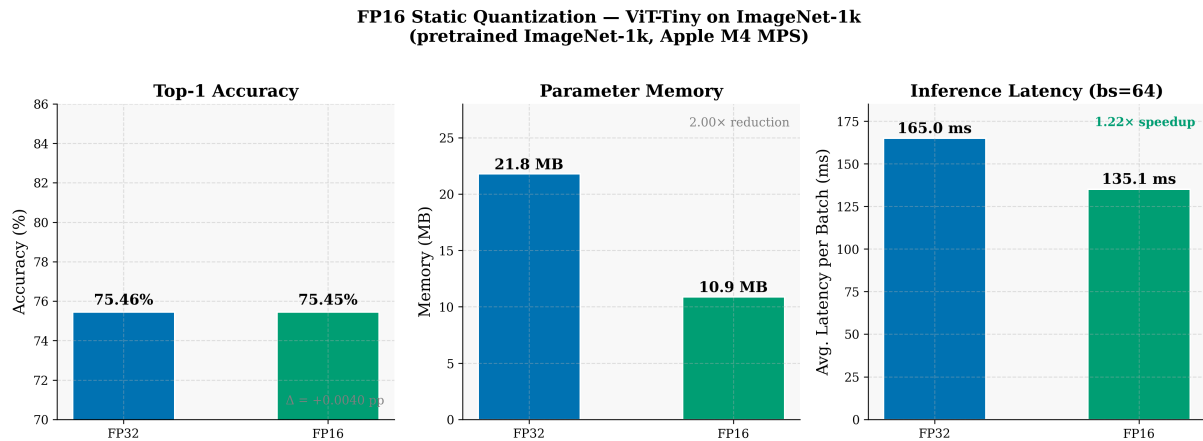


Figura 1: Overview Faza 1: acuratețe, memorie și latență — FP32 vs. FP16. Degradarea de acuratețe este invizibilă (-0.004 pp), reducerea de memorie este exactă ($2.00\times$), iar speedup-ul pe MPS este de $1.22\times$.

3.2 Acuratețe

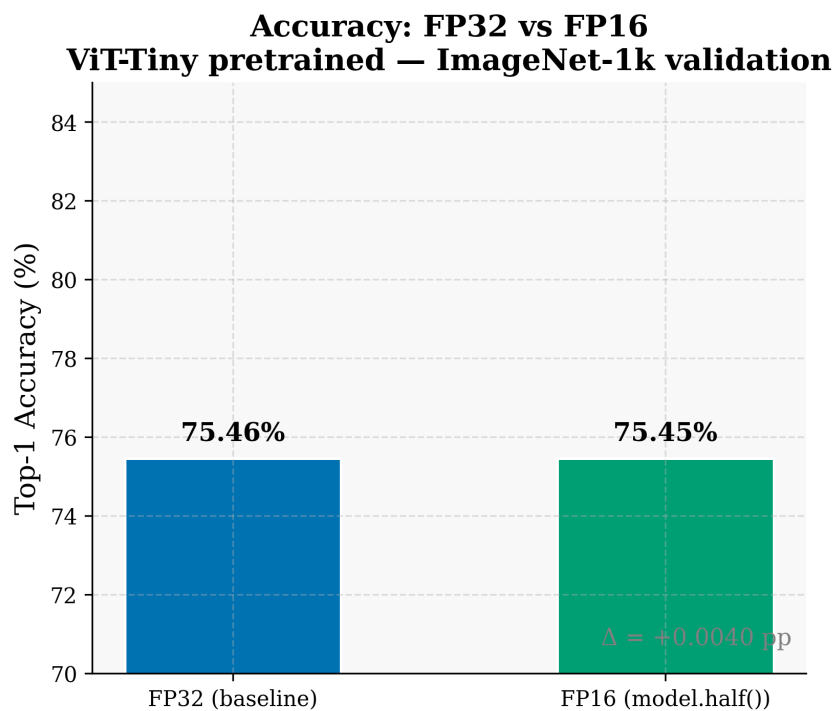


Figura 2: Comparație acuratețe Top-1: FP32 (75.456%) vs. FP16 (75.452%). Diferența de -0.004 pp este sub pragul de variabilitate numerică și confirmă că `model.half()` nu introduce degradare practică a acurateții pe ImageNet-1k.

Degradarea de -0.004 pp este neglijabilă. Aceasta reprezintă o diferență de doar **2 imagini** din 50 000 clasificate diferit. Loss-ul mediu este practic identic (0.9420 vs. 0.9420), confirmând că distribuția softmax nu este alterată semnificativ de reducerea preciziei.

3.3 Memorie

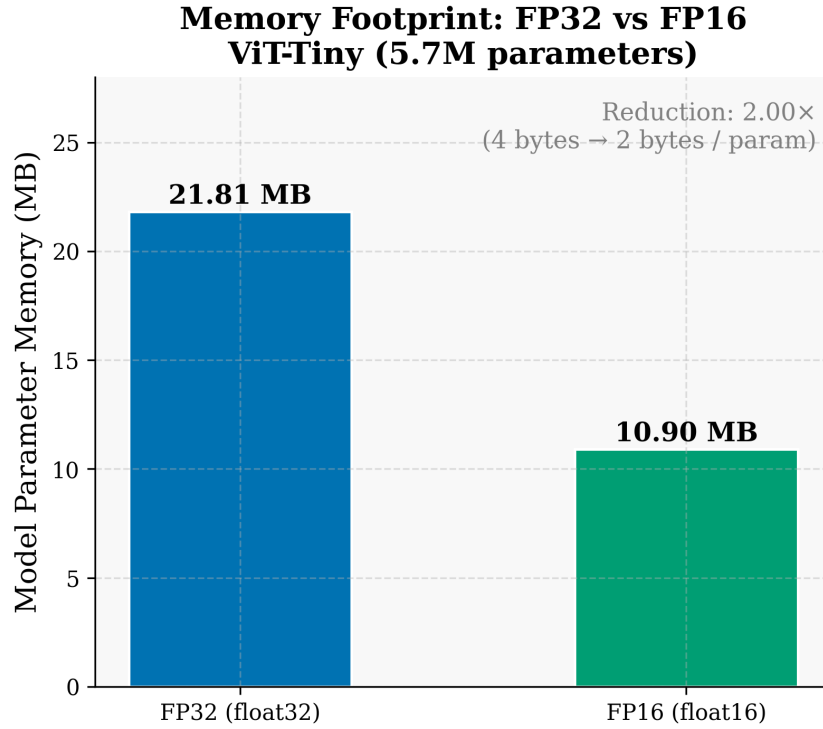


Figura 3: Footprint memorie parametri: FP32 (21.81 MB) vs. FP16 (10.91 MB). Reducerea este exact 2.00×, conform teoriei (4 bytes/param → 2 bytes/param).

Reducerea de memorie de **2.00×** este *exact* conformă cu teoria:

$$\frac{21.81 \text{ MB}}{10.91 \text{ MB}} = 2.000 \times \quad (4 \text{ bytes/param} \rightarrow 2 \text{ bytes/param}) \quad (1)$$

Pentru un model mai mare (e.g., ViT-Base cu 86M parametri), acest factor de 2× ar traduce într-o economie de ≈ 330 MB — semnificativ pentru deployment pe dispozitive edge.

3.4 Latență

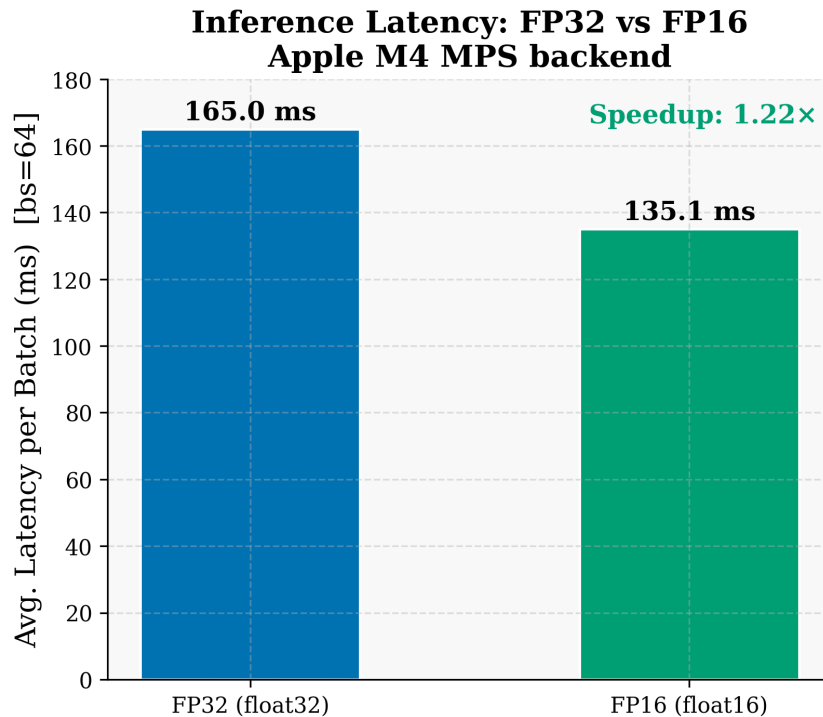


Figura 4: Latență de inferență: FP32 (164.96 ms/batch) vs. FP16 (135.14 ms/batch). Speedup de 1.22× pe Apple M4 MPS.

Speedup-ul de **1.22×** este consistent cu arhitectura Apple M4:

- Apple Silicon are suport nativ pentru operații `float16`, cu unități ALU dedicate.
- Reducerea de bandwidth (jumătate din datele transferate între memorie și compute) contribuie direct la speedup.
- Speedup-ul sub 2× este așteptat, deoarece nu toate operațiile beneficiază de FP16 (softmax, layer norm pot rămâne în FP32 intern).

3.5 Tradeoff acuratețe–latență

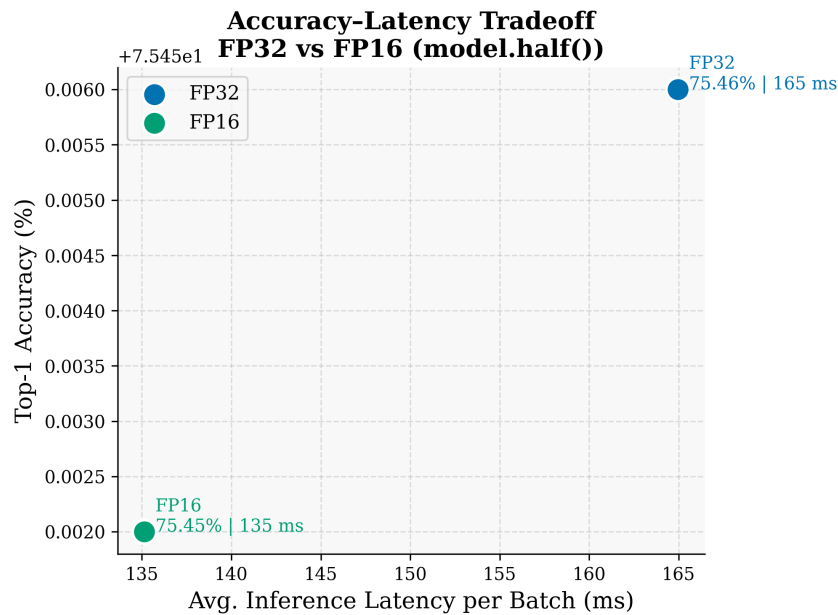


Figura 5: Scatter plot acuratețe vs. latență. FP16 se află în colțul ideal (acuratețe menținută, latență redusă).

Graficul de tradeoff confirmă că FP16 domină strict pe axa eficienței: acuratețea este practic identică, iar latența este redusă cu 30 ms/batch. Nu există niciun compromis practic pentru a adopta FP16 pe acest model.

4 Discuție

4.1 Validarea ipotezei

Rezultatele confirmă ipoteza formulată în studiul preliminar (Faza 0): **ViT-Tiny este robust la reducerea preciziei de la FP32 la FP16**. Degradarea de -0.004 pp este cu un ordin de mărime mai mică decât variabilitatea inter-run tipică (≈ 0.1 pp).

4.2 Comparatie cu studiul preliminar

Tabela 3: Comparatie: Faza 0 (CIFAR-10) vs. Faza 1 (ImageNet-1k)

Metrică	Faza 0 (CIFAR-10)	Faza 1 (ImageNet-1k)
Accuracy FP32	82.88% (fine-tuned)	75.456% (pretrained)
Degradare FP16	< 0.5 pp	-0.004 pp
Reducere memorie	$2.00\times$	$2.00\times$
Dataset	10 000 val images	50 000 val images
Clase	10	1 000

Faza 1 confirmă pe un dataset mai complex (1 000 clase, 50 000 imagini) ceea ce Faza 0 a sugerat pe CIFAR-10: **conversia FP16 este sigură și fără compromisuri practice**.

4.3 Implicații pentru fazele ulterioare

Acuratețea FP32 de **75.456%** devine baseline-ul de referință pentru toate fazele ulterioare:

- Faza 2 va evalua cuantizarea INT8 (weight-only, per-tensor) față de acest baseline.
- Faza 3 va extinde la INT8 per-channel și va realiza analiza de sensibilitate per strat.
- Orice degradare < 1 pp față de 75.456% este considerată acceptabilă pentru o reducere de memorie semnificativă.

4.4 Limitări

1. **Hardware-specific:** Speedup-ul de $1.22\times$ este specific Apple M4 MPS. Pe GPU-uri NVIDIA cu Tensor Cores, speedup-ul poate fi semnificativ mai mare (până la $2-3\times$).
2. **Un singur model:** Experimentul evaluează doar ViT-Tiny. Modele mai mari (ViT-Base, ViT-Large) ar putea beneficia mai mult de reducerea de bandwidth.
3. **Batch size fix:** Batch-ul de 64 este limitat de memoria disponibilă. Cu FP16, ar fi posibil un batch size mai mare, ceea ce ar putea îmbunătăți throughput-ul total.
4. **Procese de fundal:** Măsurătorile de latență au fost efectuate cu alte procese active (browser, Discord), ceea ce poate introduce variabilitate. Totuși, raportul de speedup rămâne valid deoarece ambele configurații au fost evaluate în aceleași condiții.

5 Concluzii

1. **Degradare neglijabilă:** FP16 introduce o pierdere de acuratețe de doar **-0.004** pp (2 imagini din 50 000), practic invizibilă.
2. **Reducere de memorie exactă:** **$2.00\times$** , conform cu teoria (`float32` $\rightarrow$ `float16`: 4 bytes $\rightarrow$ 2 bytes per parametru).
3. **Speedup pe Apple M4:** **$1.22\times$** reducere a latenței de inferență, datorită suportului nativ float16 și reducerii de bandwidth.
4. **Recomandare:** `model.half()` este o optimizare *fără cost* pentru ViT-Tiny pe ImageNet-1k — poate fi aplicată universal ca prim pas de optimizare, înaintea tehnicilor mai agresive (INT8, pruning, distillation).
5. **Baseline stabil:** Acuratețea FP32 de 75.456% pe ImageNet-1k validation devine referința pentru evaluarea cuantizării INT8 în Faza 2.

Bibliografie

- [1] A. Dosovitskiy, L. Beyer, A. Kolesnikov et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” *ICLR*, 2021. [arXiv:2010.11929](#).
- [2] H. Touvron, M. Cord, M. Douze et al., “Training data-efficient image transformers & distillation through attention,” *ICML*, 2021. [arXiv:2012.12877](#).

- [3] B. Jacob, S. Kligys, B. Chen et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” *CVPR*, 2018. [arXiv:1712.05877](#).
- [4] R. Wightman, “PyTorch Image Models (timm),” 2019. <https://github.com/rwightman/pytorch-image-models>.
- [5] O. Russakovsky, J. Deng, H. Su et al., “ImageNet Large Scale Visual Recognition Challenge,” *IJCV*, vol. 115, no. 3, pp. 211–252, 2015. [arXiv:1409.0575](#).